МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования (ИПТИП)

Кафедра индустриального программирования (ИП)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Программирование корпоративных индустриальных систем»

Практическое задание № 2

Студент группы ЭФМО-02-25 Бондарь А.Р. _____
(подпись)

Доцент Макиевский С.Е. _____
(подпись)

Отчет предоставлен «____»_____ 2026г

Москва 2026

СОДЕРЖАНИЕ

Введение 4
1 Моделирование требований и статической структуры 5
1.1 Границы системы и акторы 5
1.2 Диаграмма прецедентов (основной контур библиотеки) 6
1.3 Сценарии ключевых прецедентов 8
1.3.1 Регистрация нового читателя 8
1.3.2 Выдача книги читателю 9
1.3.3 Возврат книги 10
1.3.4 Оплата штрафа 10
1.3.5 Управление пользователями 11
1.3.6 Поиск книги в каталоге 12
1.3.7 Формирование отчёта о задолжниках 12
1.4 Словарь терминов предметной области 13
1.5 Доменная модель (диаграмма классов) 13
1.5.1 Классы и их атрибуты 14
1.5.2 Связи и кратности 15
1.5.3 Ограничения (инварианты) 16
1.6 Структурные ограничения и валидаторы 16
1.6.1 Бизнес-правила, контролируемые на уровне БД 16
1.6.2 Бизнес-правила, контролируемые на уровне приложения 16
1.6.3 Валидаторы входных данных 17
1.7 Диаграмма пакетов и правила зависимостей 17
1.8 Матрица трассируемости требований к моделям 18
2 Моделирование взаимодействий 19
2.1 Выбор сценариев и границ моделирования 19
2.2 Участники взаимодействия 20
2.2.1 Общие участники для всех сценариев 20

2.2.2 Ответственность 20
2.2.3 Запрет прямых зависимостей 21
2.3 Диаграммы кооперации по сценариям 21
2.3.1 Сценарий «Выдача книги» 21
2.3.2 Сценарий «Возврат книги» 23
2.3.3 Сценарий «Регистрация читателя» 24
2.4 Таблица сообщений и контрактов 25
2.4.1 Сообщение: `LoanService.issueBook(readerId, copyId)` 25
2.4.2 Сообщение: `ReturnService.returnBook(copyId)` 25
2.4.3 Сообщение: `ReaderService.register(readerData)` 26
2.5 Границы слоёв и точки входа 26
2.6 Границы атомарных операций 27
2.7 Обработка ошибок и альтернативные ветви 27
2.7.1 Типовые ошибки и реакции 27
2.8 Сопоставление взаимодействий с прецедентами и классами 28
2.9 Требования к наблюдаемости 29
2.9.1 События для логирования (аудит) 29
2.9.2 Метрики 29
2.9.3 Корреляционный идентификатор 29
Заключение 30

ВВЕДЕНИЕ

Данный документ содержит результаты выполнения практических занятий №5 и №6 по дисциплине «Программирование корпоративных индустриальных систем» на примере автоматизации работы библиотеки (учёт книг, читателей, выдачи и возврата изданий). Представлены диаграммы прецедентов, сценарии, словарь терминов, доменная модель, диаграмма пакетов, матрица трассировки, а также диаграммы кооперации для трёх ключевых сценариев с описанием контрактов, слоёв, обработки ошибок и наблюдаемости.

1 МОДЕЛИРОВАНИЕ ТРЕБОВАНИЙ И СТАТИЧЕСКОЙ СТРУКТУРЫ

1.1 Границы системы и акторы

Границы системы:

Автоматизированная система библиотеки охватывает:

- учёт книжного фонда (книги, экземпляры);
- регистрацию и ведение картотеки читателей;
- процессы выдачи, возврата и продления книг;
- начисление и учёт штрафов;
- формирование отчётности;
- управление справочниками (авторы, жанры и т.п.).

Внешняя среда:

- читатели (пользователи);
- библиотекари;
- администраторы;
- внешние системы (электронный каталог, платёжный шлюз, система уведомлений).

Акторы:

Таблица 1 – Акторы системы

Актор	**Описание**	**Права**	**Аудит действий**
Читатель	Зарегистрированный пользователь библиотеки. Может искать книги, просматривать свой профиль, историю выдач, продлевать книги (если разрешено).	Только чтение своего профиля, поиск	Нет
Библиотекарь	Сотрудник библиотеки. Выполняет выдачу/возврат, регистрирует читателей, пополняет каталог, управляет справочниками.	Полные права на операции с книгами, читателями, выдачами	Да (все действия)

Продолжение Таблицы 1

Актор	**Описание**	**Права**	**Аудит действий**
Администратор	Управляет правами доступа, просматривает логи аудита, настраивает параметры системы (сроки выдачи, лимиты, штрафы).	Все права, включая управление пользователями системы	Да
Внешняя платёжная система	Принимает запросы на оплату штрафов (интеграция).	Только приём платежей	Нет (внешняя)
Система уведомлений	Отправляет email/SMS читателям о приближении срока возврата, задолженностях.	Только отправка	Нет

1.2 Диаграмма прецедентов (основной контур библиотеки)

Акторы:

– Читатель - поиск книг, просмотр истории, продление (если разрешено);

– Библиотекарь - управление каталогом, регистрация читателя, выдача, возврат, формирование отчётов;

– Администратор - управление пользователями, настройки, просмотр аудита;

– Внешняя платёжная система;

– Система уведомлений.

Прецеденты:

– Управление каталогом (добавление/удаление/редактирование книг и экземпляров);

– Поиск книг (по автору, названию, ISBN, жанру);

– Просмотр истории/профиля;

– Регистрация читателя;

– Выдача книги читателю;

– Возврат книги;

– Продление срока выдачи;

– Начисление штрафа (при просрочке);

– Оплата штрафа (через внешнюю систему);

– Формирование отчётов (задолжники, популярные книги, статистика выдач);

– Управление пользователями (для администратора).

Связи:

– «Выдача книги» включает проверку наличия книги и статуса читателя (`include`);

– «Возврат книги» может включать расчёт штрафа (`extend` при просрочке);

– «Продление срока» расширяет выдачу при соблюдении условий;

– «Оплата штрафа» использует внешнюю платёжную систему.

Связи акторов с прецедентами

– Читатель: Поиск книг, Просмотр истории, Продление срока (если разрешено), Возврат книги (через терминал самообслуживания), Оплата штрафа;

– Библиотекарь: Управление каталогом, Регистрация читателя, Выдача книги, Возврат книги (приём от читателя), Формирование отчётов;

– Администратор: Управление пользователями, а также может иметь доступ к настройкам системы и аудиту (можно выделить отдельно «Управление системой»);

– Внешняя платёжная система: участвует в прецеденте «Оплата штрафа» (связь типа «include» или просто ассоциация).

Связи между прецедентами (include/extend):

– «Выдача книги» include «Проверка статуса читателя» и «Проверка наличия экземпляра» (эти проверки можно показать как отдельные прецеденты, но часто они остаются внутри);

– «Возврат книги» include «Расчёт штрафа» (внутренняя операция);

– «Возврат книги» extend -> «Оплата штрафа» (условие: наличие штрафа и желание читателя оплатить сразу);

– «Продление срока» extend «Выдача книги» (условие: не истёк срок, нет очереди и т.д.);

– «Оплата штрафа» include «Взаимодействие с платёжной системой» (можно показать как отдельный прецедент, но для простоты оставляем внутри).

Рисунок 1 – Диаграмма прецедентов

1.3 Сценарии ключевых прецедентов

1.3.1 Регистрация нового читателя

Основной поток:

1. Библиотекарь выбирает функцию «Регистрация читателя».
2. Система запрашивает персональные данные: ФИО, адрес, телефон, email, паспортные данные.
3. Библиотекарь вводит данные.
4. Система проверяет уникальность читателя (по паспорту или email).
5. Система создает запись читателя, присваивает уникальный номер читательского билета.
6. Система выводит подтверждение с данными читателя.

Альтернативы:

– 4а. Читатель уже существует -> уведомление, предложение активировать старую запись или обновить данные.

– 3а. Не заполнены обязательные поля -> повторный запрос с подсветкой ошибок.

– Ошибка БД -> логирование, сообщение о временной недоступности.

1.3.2 Выдача книги читателю

Основной поток:

1. Библиотекарь сканирует штрих-код экземпляра книги (или вводит его идентификатор).
2. Система проверяет наличие экземпляра (статус «в наличии»).
3. Библиотекарь вводит номер читательского билета.
4. Система проверяет статус читателя: отсутствие задолженностей, лимит выдач не превышен.
5. Система фиксирует выдачу: создает запись о выдаче, меняет статус экземпляра на «выдана», записывает дату выдачи и планируемую дату возврата (текущая дата + срок).
6. Система выводит подтверждение.

Альтернативы:

– 2а. Экземпляр не найден или не в наличии -> сообщение об ошибке.

– 4а. Читатель имеет задолженности -> отказ, предложение оплатить штраф.

– 4б. Превышен лимит выдач -> отказ.

– 4в. Читатель заблокирован -> отказ.

1.3.3 Возврат книги

Основной поток:

1. Актор (библиотекарь или читатель) инициирует возврат, сканируя штрих-код экземпляра (или вводя его идентификатор).
2. Система находит активную выдачу по этому экземпляру.
3. Система вычисляет дату возврата и сравнивает с плановой. Если просрочка, рассчитывает штраф.
4. Система меняет статус экземпляра на «в наличии», закрывает выдачу (заполняет фактическую дату возврата).
5. Если есть штраф, система создаёт запись о штрафе и оповещает актора.
6. Система выводит информацию о возврате и сумме штрафа (если есть).

Альтернативы:

– 2а. Активная выдача не найдена -> запрос на проверку (возможно, ошибка сканирования).

– 5а. Если штраф начислен, читатель может сразу перейти к оплате. Система предлагает оплатить штраф (переход к прецеденту «Оплата штрафа» с передачей данных о штрафе). После успешной оплаты (или отказа) возврат считается завершённым.

– 5б. Читатель оплачивает позже (через личный кабинет) – не влияет на текущий поток.

1.3.4 Оплата штрафа

Основной поток:

1. Читатель выбирает в личном кабинете неоплаченный штраф (или переходит по предложению после возврата).
2. Система отображает детали штрафа (сумма, дата начисления, книга).
3. Читатель подтверждает оплату.

4. Система передаёт запрос во внешнюю платёжную систему (сумма, идентификатор штрафа, данные читателя).

5. Платёжная система обрабатывает платёж и возвращает результат (успех/отказ).

6. При успехе система обновляет статус штрафа на «оплачен», фиксирует дату оплаты.

7. Система отображает читателю подтверждение оплаты.

Альтернативы:

– 5а. Платёж отклонён (недостаточно средств, ошибка карты) -> уведомление читателя, предложение повторить или выбрать другой способ;

– 5б. Платёжная система недоступна -> сообщение о временной недоступности, штраф остаётся неоплаченным (можно повторить позже);

– 4а. Читатель отменяет оплату на любом шаге -> возврат к списку штрафов.

1.3.5 Управление пользователями

Основной поток:

1. Администратор входит в раздел управления пользователями.

2. Система отображает список пользователей (библиотекарей, читателей) с фильтрами.

3. Администратор выбирает действие:

– Создание нового пользователя (задание роли, логина, пароля).

– Редактирование данных существующего пользователя.

– Блокировка/разблокировка учётной записи.

– Назначение ролей (библиотекарь, администратор).

4. Система применяет изменения и фиксирует в журнале аудита.

5. Система подтверждает выполнение.

Альтернативы:

– 3а. При создании пользователя с уже существующим логином -> ошибка, запрос на другой логин.

– 3б. При попытке заблокировать самого себя -> защита, отказ.

1.3.6 Поиск книги в каталоге

Основной поток:

1. Читатель вводит критерии поиска (название, автор, ISBN, жанр).
2. Система выполняет поиск по каталогу книг (без учёта экземпляров).
3. Система отображает список найденных книг с указанием наличия экземпляров (в библиотеке или выданы).
4. Читатель может выбрать книгу для просмотра подробной информации (авторы, издательство, аннотация).

Альтернативы:

– 2а. Ничего не найдено -> сообщение «Книги не найдены».

– Ошибка БД -> временная недоступность.

1.3.7 Формирование отчёта о задолжниках

Основной поток:

1. Библиотекарь выбирает отчёт «Задолжники».
2. Система запрашивает параметры: период просрочки (дни) или дату актуальности.
3. Система формирует список читателей, у которых есть невозвращённые книги с просроченной датой возврата, с указанием книг и количества дней просрочки.
4. Система выводит отчёт на экран или позволяет экспортировать в PDF/Excel.

Альтернативы:

– Нет задолженностей -> пустой отчёт.

– Ошибка выборки -> сообщение.

1.4 Словарь терминов предметной области

Таблица 2 – Словарь терминов

Термин	**Определение**
Книга (издание)	Описательная запись о произведении печати (ISBN, название, автор, издательство и т.д.). Не привязана к физическому экземпляру.
Экземпляр книги	Физический носитель книги с уникальным инвентарным номером (штрих-код). Имеет статус (в наличии, выдана, списана).
Читатель	Физическое лицо, зарегистрированное в системе и имеющее право брать книги.
Читательский билет	Уникальный идентификатор читателя в системе.
Выдача	Факт передачи экземпляра книги читателю на определённый срок. Содержит дату выдачи, планируемую дату возврата, фактическую дату возврата.
Штраф	Денежное взыскание за просрочку возврата или утерю книги.
Задолженность	Наличие у читателя активной выдачи с датой возврата ранее текущей даты.
Лимит выдач	Максимальное количество книг, которое читатель может держать одновременно.
Каталог	Совокупность записей о книгах (без привязки к экземплярам).
Статус экземпляра	Состояние: в наличии, выдана, на ремонте, списана.
Статус читателя	Активен, заблокирован (за долги), удалён.

1.5 Доменная модель (диаграмма классов)

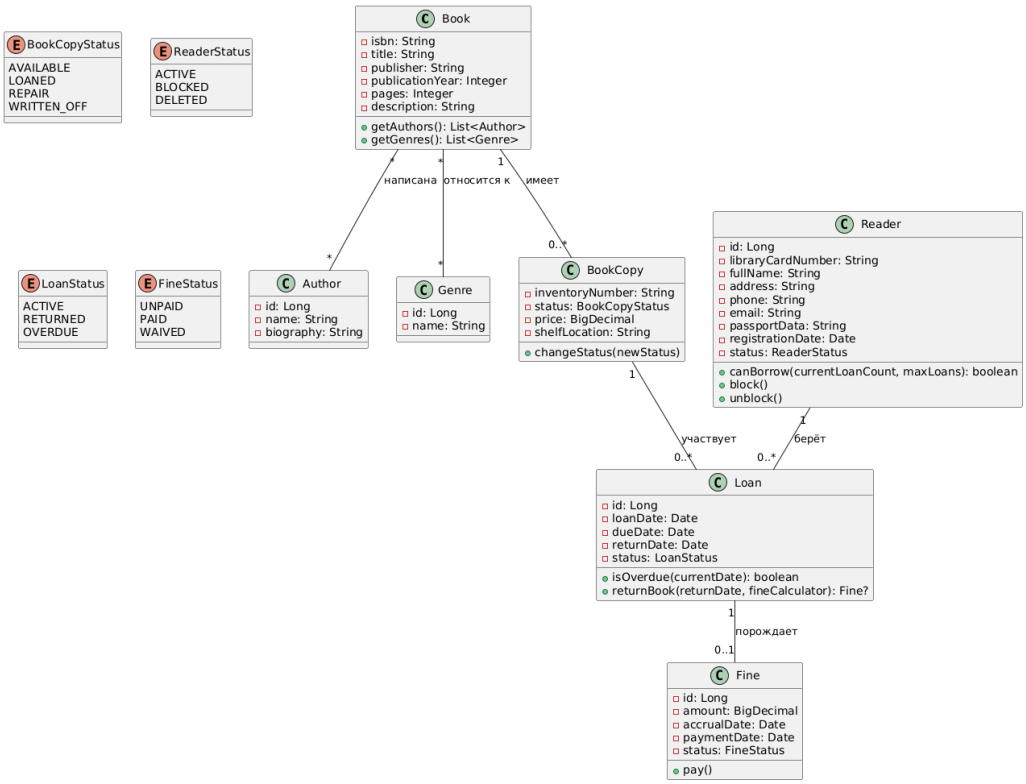

Рисунок 2 – Диаграмма классов

1.5.1 Классы и их атрибуты

Класс `Book` (книга)

- `isbn: String` (уникальный)
- `title: String`
- `publisher: String`
- `publicationYear: Integer`
- `pages: Integer`
- `description: String`
- `+ getAuthors(): List<Author>`
- `+ getGenres(): List<Genre>`

Класс `Author`

- `id: Long`
- `name: String`
- `biography: String`

Класс `Genre`

- `id: Long`
- `name: String`

Класс `BookCopy` (экземпляр книги)

- `inventoryNumber: String` (уникальный)
- `status: BookCopyStatus` (enum: AVAILABLE, LOANED, REPAIR, WRITTEN_OFF)
- `price: BigDecimal`
- `shelfLocation: String`
- `+ changeStatus(newStatus)`

Класс `Reader`

- `id: Long`
- `libraryCardNumber: String` (уникальный)
- `fullName: String`

– `address: String`

– `phone: String`

– `email: String`

– `passportData: String` (уникальный)

– `registrationDate: Date`

– `status: ReaderStatus` (ACTIVE, BLOCKED, DELETED)

– `+ canBorrow(currentLoanCount: int, maxLoans: int): boolean`

– `+ block()`

– `+ unblock()`

Класс `Loan` (выдача)

– `id: Long`

– `loanDate: Date`

– `dueDate: Date`

– `returnDate: Date` (null, если не возвращена)

– `status: LoanStatus` (ACTIVE, RETURNED, OVERDUE)

– `+ isOverdue(currentDate): boolean`

– `+ returnBook(returnDate: Date, fineCalculator): Fine?`

Класс `Fine` (штраф)

– `id: Long`

– `amount: BigDecimal`

– `accrualDate: Date`

– `paymentDate: Date` (null, если не оплачен)

– `status: FineStatus` (UNPAID, PAID, WAIVED)

– `+ pay()`

1.5.2 Связи и кратности

– `Book` (1) — (0..*) `BookCopy` (одна книга может иметь много экземпляров)

– `Book` (*) — (*) `Author` (книгу могут написать несколько авторов)

– `Book` (*) — (*) `Genre` (книга может относиться к нескольким жанрам)

– `Reader` (1) — (0..*) `Loan` (читатель может иметь много выдач)

– `BookCopy` (1) — (0..*) `Loan` (экземпляр может участвовать во многих выдачах, но одновременно только одна активная)

– `Loan` (1) — (0..1) `Fine` (к выдаче может быть привязан штраф)

1.5.3 Ограничения (инварианты)

1. У экземпляра не может быть более одной активной выдачи (статус `LOANED`).
2. Читатель не может иметь активных выдач больше лимита.
3. Дата возврата не может быть раньше даты выдачи.
4. Штраф начисляется только если фактическая дата возврата > планируемой.
5. Статус читателя `BLOCKED` запрещает выдачу.
6. Инвентарный номер экземпляра уникален.
7. ISBN книги уникален (в пределах системы).
8. Номер читательского билета и паспортные данные уникальны.

1.6 Структурные ограничения и валидаторы

1.6.1 Бизнес-правила, контролируемые на уровне БД

– Уникальность `inventoryNumber` в `BookCopy`

– Уникальность `isbn` в `Book`

– Уникальность `libraryCardNumber` и `passportData` в `Reader`

– Внешние ключи обеспечивают ссылочную целостность

1.6.2 Бизнес-правила, контролируемые на уровне приложения

– Проверка лимита выдач перед созданием `Loan`

– Проверка статуса читателя (не заблокирован)

– Проверка статуса экземпляра (`AVAILABLE`)

– Расчёт штрафа по формуле (количество дней просрочки * дневной тариф)

– Проверка, что экземпляр не выдан дважды одновременно (через поиск активной выдачи)

1.6.3 Валидаторы входных данных

– Формат email

– Контрольная цифра ISBN (алгоритм 10 или 13)

– Номер телефона по маске

– Неотрицательная сумма штрафа

1.7 Диаграмма пакетов и правила зависимостей

Пакеты (слои):

– **Presentation** — контроллеры REST, обработчики команд, представления (зависит от Application);

– **Application** — сервисы приложения, DTO, мапперы, интерфейсы репозиториев (зависит от Domain);

– **Domain** — сущности, value objects, доменные сервисы, события. Не зависит от других слоёв;

– **Infrastructure** — реализации репозиториев, UnitOfWork, логирование, отправка email, доступ к БД (зависит от Domain и может от Application через интерфейсы).

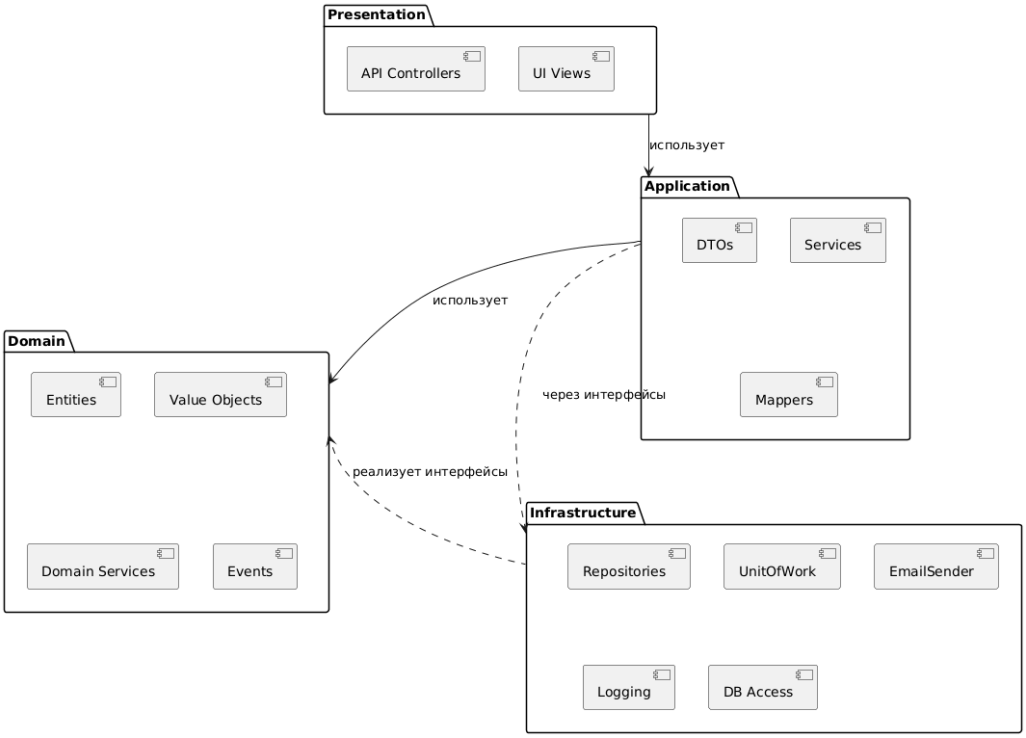

Рисунок 3 – Диаграмма пакетов

Правила зависимостей заключаются в строгом направление от верхних слоёв к нижним, запрещены циклические зависимости.

1.8 Матрица трассируемости требований к моделям

Таблица 3 – Матрица трассировки

Требование	**Прецедент**	**Элемент модели**	**Способ проверки**
Регистрация читателя с уникальным номером	Регистрация читателя	Reader (libraryCardNumber)	Уникальный индекс в БД
Проверка статуса читателя перед выдачей	Выдача книги	Reader.status, метод canBorrow()	Модульный тест
Учёт экземпляров книг с уникальным инвентарным номером	Управление каталогом	BookCopy.inventoryNumber	Уникальный индекс
Фиксация даты выдачи и срока возврата	Выдача книги	Loan.loanDate, Loan.dueDate	Интеграционный тест
Расчёт штрафа при просрочке	Возврат книги	Fine, метод Loan.returnBook()	Тест с просрочкой
Блокировка читателя при задолженностях	(неявно)	Reader.status, Job по просрочкам	Тест задания

2 МОДЕЛИРОВАНИЕ ВЗАИМОДЕЙСТВИЙ

2.1 Выбор сценариев и границ моделирования

Сценарий 1: Выдача книги читателю

Начальные условия: читатель зарегистрирован, экземпляр книги в наличии.

Входные данные: идентификатор читателя (`readerId`), идентификатор экземпляра (`copyId`).

Конечный результат: создана запись о выдаче, статус экземпляра = `LOANED`.

Моделируется внутреннее взаимодействие (проверки, создание, сохранение). Внешняя интеграция — отправка уведомления читателю (после коммита).

Сценарий 2: Возврат книги

Начальные условия: экземпляр выдан (активная выдача).

Входные данные: идентификатор экземпляра (`copyId`).

Конечный результат: выдача закрыта, экземпляр в наличии, при просрочке создан штраф.

Моделируется внутреннее взаимодействие, включая расчёт штрафа. Интеграция с платёжной системой не входит в сценарий (может быть отдельно).

Сценарий 3: Регистрация нового читателя

Начальные условия: нет.

Входные данные: персональные данные читателя.

Конечный результат: создана запись читателя, присвоен номер билета.

Моделируется только внутреннее взаимодействие; проверка паспортных данных через внешний сервис (опционально) — может быть интеграцией.

2.2 Участники взаимодействия

2.2.1 Общие участники для всех сценариев

– **Контроллер** (`LibraryController`) — точка входа, принимает HTTP-запросы.

– **Сервис приложения** (например, `LoanService`, `ReaderService`) — оркестрация сценария.

– **Репозитории** (интерфейсы) — `ReaderRepository`, `BookCopyRepository`, `LoanRepository`, `FineRepository`.

– **Доменные объекты** — `Reader`, `BookCopy`, `Loan`, `Fine`.

– **UnitOfWork** — управление транзакцией.

– **Logger** — запись событий аудита.

– **EventBus** — публикация доменных событий (например, `BookLoaned`, `BookReturned`, `FineCharged`).

– **EmailSender** — инфраструктурный компонент для уведомлений.

2.2.2 Ответственность

Таблица 4 – Ответственность участников

Участник	**Ответственность**
Контроллер	Преобразование входных данных, вызов сервиса, формирование ответа
Сервис приложения	Получение данных из репозиториев, вызов методов доменных объектов, сохранение, управление транзакцией
Репозитории	Доступ к хранилищу (поиск, добавление, обновление)
Доменные объекты	Реализация бизнес-правил, изменение состояния
UnitOfWork	Обеспечение атомарности операций
Logger	Запись событий и ошибок
EmailSender	Отправка уведомлений (после успешного завершения)

2.2.3 Запрет прямых зависимостей

Доменные объекты не знают о репозиториях, UnitOfWork, сервисах приложения. Сервис приложения зависит от интерфейсов репозиториев, но не от их конкретных реализаций. Инфраструктурные компоненты (EmailSender) вызываются сервисом приложения после завершения транзакции (через события или явно).

2.3 Диаграммы кооперации по сценариям

2.3.1 Сценарий «Выдача книги»

Участники: `controller: LibraryController`, `loanService: LoanApplicationService`, `readerRepo: ReaderRepository`, `bookCopyRepo: BookCopyRepository`, `loanRepo: LoanRepository`, `unitOfWork: UnitOfWork`, `logger: Logger`, `reader: Reader` (объект домена), `bookCopy: BookCopy`, `loan: Loan`.

Сообщения (нумерованы):

1. `controller.issueBook(readerId, copyId)` -> `loanService.issueBook`
2. `loanService` -> `readerRepo.findById(readerId)` -> возвращает `reader`
3. `loanService` -> `bookCopyRepo.findById(copyId)` -> возвращает `bookCopy`
4. `loanService` -> `loanRepo.countActiveByReader(readerId)` -> возвращает `activeLoansCount`
5. `loanService` вызывает `reader.canBorrow(activeLoansCount, MAX_LOANS)` -> проверка лимита
6. `loanService` проверяет `bookCopy.isAvailable()` -> если статус не AVAILABLE, ошибка

7. `loanService` создаёт `loan = new Loan(reader, bookCopy, loanDate, dueDate)` (в конструкторе `loan` меняет статус `bookCopy` на LOANED)

8. `loanService` -> `loanRepo.add(loan)`

9. `loanService` -> `unitOfWork.commit()`

10. `loanService` публикует событие `BookLoaned` (через EventBus)

11. `loanService` возвращает результат `controller`

12. (после ответа) слушатель события отправляет email через `emailSender`

Альтернативные потоки:

– Если `reader` не найден — ошибка `ReaderNotFoundException`

– Если `bookCopy` не найден — `BookCopyNotFoundException`

– Если читатель заблокирован — `ReaderBlockedException`

– Если превышен лимит — `LoanLimitExceededException`

– Если книга не доступна — `BookNotAvailableException`

– Ошибка БД при коммите — откат транзакции, логирование

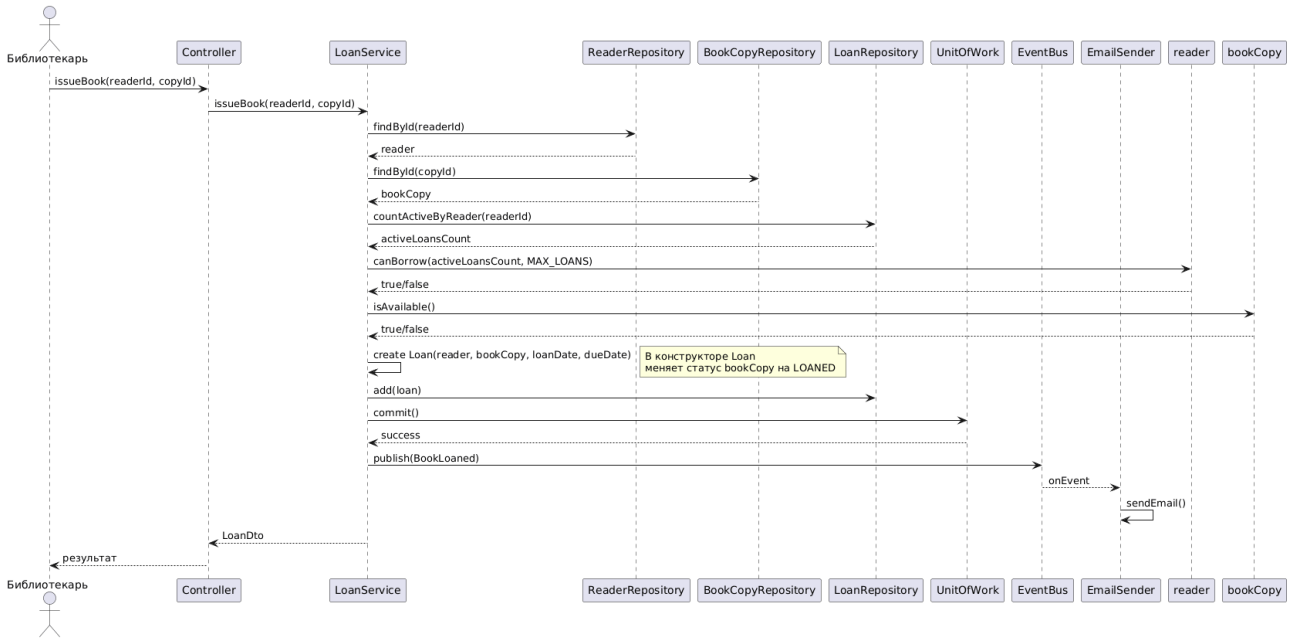

Рисунок 4 – Диаграмма последовательности. Выдача книги

2.3.2 Сценарий «Возврат книги»

Участники: `controller: LibraryController`, `returnService: ReturnApplicationService`,`bookCopyRepo: BookCopyReposito` `loanRepo: LoanRepository`, `fineRepo: FineRepository`, `unitOfWork UnitOfWork`, `logger: Logger`, `bookCopy: BookCopy`, `loan: Loan`. `fine` (опционально).

Сообщения:

1. `controller.returnBook(copyId)` -> `returnService.returnBook`
2. `returnService` -> `bookCopyRepo.findById(copyId)` -> `bookCopy`
3. `returnService` -> `loanRepo.findActiveByCopy(copyId)` -> `loan`
4. `returnService` вызывает `loan.returnBook(returnDate, fineCalculator)` Внутри `loan`:
   - вычисляет просрочку,
   - если есть — создаёт `Fine` (через фабрику или доменный сервис),
   - меняет статус `bookCopy` на AVAILABLE,
   - возвращает созданный `fine` (или null)
5. Если `fine != null`, `returnService` -> `fineRepo.add(fine)`
6. `returnService` -> `loanRepo.update(loan)`
7. `returnService` -> `unitOfWork.commit()`
8. Публикация события `BookReturned` с информацией о штрафе
9. Возврат результата `controller`

Альтернативы:

- Если активная выдача не найдена — ошибка `NoActiveLoanException`
- Ошибка при расчёте штрафа (некорректные данные) — логирование, откат

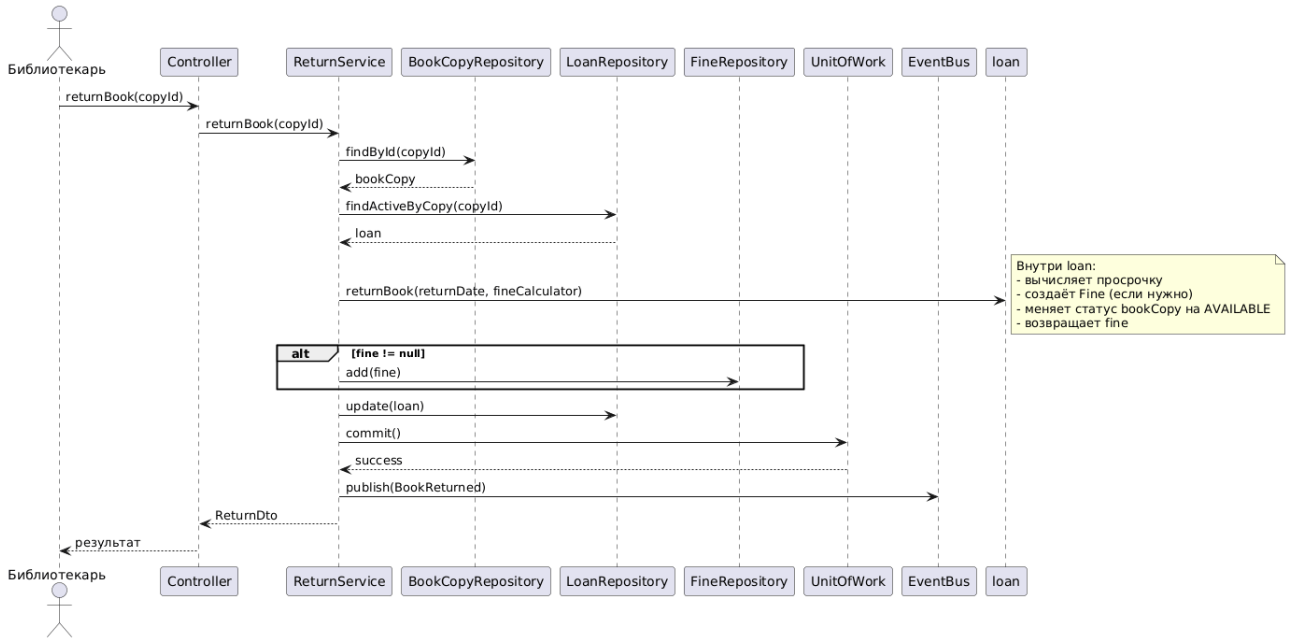

Рисунок 5 – Диаграмма последовательности: Возврат книги

2.3.3 Сценарий «Регистрация читателя»

Участники: `controller: LibraryController`, `readerService: ReaderApplicationService`, `readerRepo: ReaderRepository`, `unitOfWork UnitOfWork`, `logger: Logger`,

Сообщения:

1. `controller.registerReader(readerData)` -> `readerService.register`
2. `readerService` проверяет уникальность паспорта через `readerRepo.existsByPassport(passport)`
3. Если уникально, создаёт `reader = new Reader(...)`
4. `readerService` -> `readerRepo.add(reader)`
5. `readerService` -> `unitOfWork.commit()`
6. Публикация события `ReaderRegistered`
7. Возврат созданного `reader` (или DTO)

Альтернативы:

– Читатель с таким паспортом уже существует -> `ReaderAlreadyExistsException`

– Нарушение уникальности на уровне БД -> обработка, откат

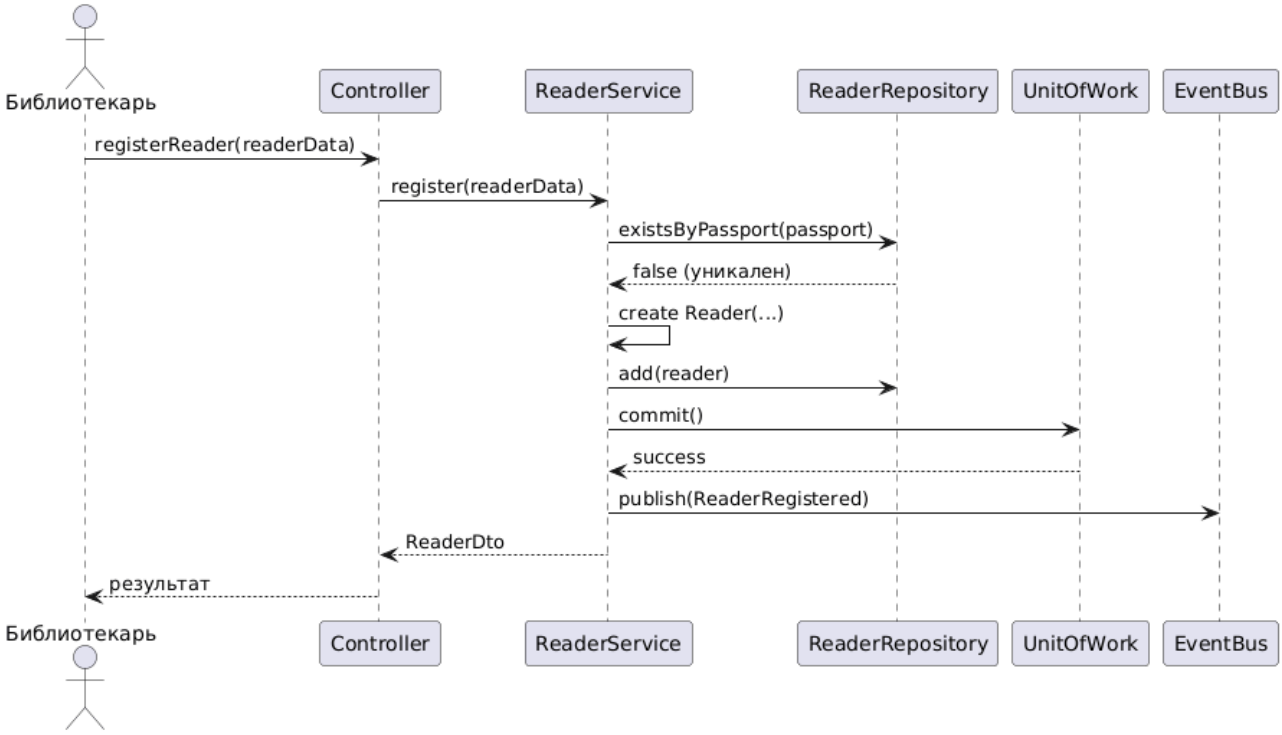

Рисунок 6 – Диаграмма последовательности. Регистрация читателя

2.4 Таблица сообщений и контрактов

2.4.1 Сообщение: `LoanService.issueBook(readerId, copyId)`

Таблица 5 – Контракт сообщения issueBook

Поле	**Значение**
Входные данные	`readerId: Long`, `copyId: Long`
Выходные данные	`LoanDto` (id, readerId, copyId, loanDate, dueDate)
Ошибки	`ReaderNotFoundException`, `BookCopyNotFoundException`, `ReaderBlockedException`, `LoanLimitExceededException`, `BookNotAvailableException`, `DatabaseException`
Идемпотентность	Повторный вызов с теми же данными должен вернуть ошибку (книга уже выдана). Можно поддержать идемпотентность через ключ идемпотентности, чтобы избежать дублирования при повторной отправке.
Аудит	Фиксируется событие `BookLoaned` с данными: librarianId, readerId, copyId, timestamp.

2.4.2 Сообщение: `ReturnService.returnBook(copyId)`

Таблица 6 – Контракт сообщения returnBook

Поле	**Значение**
Входные данные	`copyId: Long`
Выходные данные	`ReturnDto` (copyId, returnDate, fineAmount)
Ошибки	`BookCopyNotFoundException`, `NoActiveLoanException`, `DatabaseException`
Идемпотентность	Если вызов повторяется после успешного возврата, система должна вернуть ошибку (книга уже возвращена). Идемпотентность можно обеспечить проверкой статуса.
Аудит	Событие `BookReturned` с суммой штрафа.

2.4.3 Сообщение: `ReaderService.register(readerData)`

Таблица 7 – Контракт сообщения register

Поле	**Значение**
Входные данные	`ReaderCreateDto` (fullName, address, phone, email, passportData)
Выходные данные	`ReaderDto` (id, libraryCardNumber, ...)
Ошибки	`ReaderAlreadyExistsException`, `ValidationException`, `DatabaseException`
Идемпотентность	Если повторный вызов с теми же данными, система должна вернуть конфликт, т.к. читатель уже существует. Идемпотентность не требуется.
Аудит	Событие `ReaderRegistered` с данными читателя.

2.5 Границы слоёв и точки входа

Точка входа: контроллер (REST API). Он принимает запрос, валидирует входные DTO и вызывает соответствующий сервис приложения.

Границы слоёв:

– `Presentation` -> `Application`: через вызов методов сервиса с DTO;

– `Application` -> `Domain`: сервис приложения получает доменные объекты через репозитории (интерфейсы), вызывает их методы;

– `Application` -> `Infrastructure`: через интерфейсы репозиториев (реализованы в инфраструктуре), а также через `UnitOfWork` (транзакция), `EmailSender` (после коммита).

Проверка независимости домена: доменные объекты не используют аннотации ORM, не ссылаются на репозитории. Все зависимости направлены внутрь домена.

2.6 Границы атомарных операций

Транзакция охватывает все операции с БД в рамках одного сценария:

– Для выдачи: чтение читателя, экземпляра, создание выдачи, изменение статуса, сохранение;

– Для возврата: чтение экземпляра, поиск выдачи, обновление выдачи, создание штрафа, обновление статуса экземпляра;

– Для регистрации: проверка уникальности, создание читателя.

Все эти шаги должны выполняться атомарно, чтобы не возникло рассинхронизации. Транзакция начинается перед первым обращением к репозиторию и завершается коммитом (или откатом). Используется `UnitOfWork` (или `@Transactional`).

Компенсация для внешних интеграций: отправка уведомлений производится после успешного коммита, чтобы в случае ошибки отправки не откатывать транзакцию. Если уведомление не удалось, оно логируется и может быть повторено позже (фоновый процесс).

2.7 Обработка ошибок и альтернативные ветви

2.7.1 Типовые ошибки и реакции

Таблица 8 – Обработка ошибок

Ошибка	**HTTP статус**	**Сообщение пользователю**	**Действия системы**
Читатель не найден	404 Not Found	«Читатель с указанным ID не существует»	Логирование INFO
Экземпляр не найден	404	«Книга не найдена»	Логирование INFO
Читатель заблокирован	403 Forbidden	«Читатель заблокирован. Обратитесь к администратору»	Логирование WARN
Превышен лимит выдач	400 Bad Request	«Читатель превысил лимит выдач»	Логирование INFO
Книга не доступна	409 Conflict	«Данный экземпляр книги уже выдан»	Логирование INFO
Нет активной выдачи	404	«Книга не числится выданной»	Логирование INFO

Продолжение Таблицы 8

Ошибка	**HTTP статус**	**Сообщение пользователю**	**Действия системы**
Ошибка БД (connection)	503 Service Unavailable	«Временная ошибка, повторите позже»	Логирование ERROR
Нарушение уникальности (читатель)	409 Conflict	«Читатель с такими паспортными данными уже зарегистрирован»	Логирование INFO

Предотвращение повторных ошибок:

– Идемпотентность запросов через ключи идемпотентности (например, в заголовке `Idempotency-Key`). Для выдачи и возврата повторные запросы с тем же ключом должны возвращать ранее созданный результат или ошибку;

– Блокировки на уровне БД (оптимистичные/пессимистичные) для избежания гонок (например, при параллельной выдаче одного экземпляра).

Логирование ошибок: все исключения записываются в лог с указанием correlationId, стека вызовов и входных данных (без чувствительной информации).

2.8 Сопоставление взаимодействий с прецедентами и классами

– Сценарий «Выдача книги» реализует одноимённый прецедент. Используемые классы: `Reader`, `BookCopy`, `Loan` — соответствуют доменной модели;

– Сценарий «Возврат книги» реализует прецедент «Возврат книги», использует `Loan`, `Fine`, `BookCopy`;

– Сценарий «Регистрация читателя» реализует прецедент «Регистрация читателя», использует `Reader`.

Все данные, необходимые для шагов прецедентов (статусы, лимиты, даты), присутствуют в доменных объектах. Проверки (статус читателя, доступность книги) выполняются методами доменных объектов, что гарантирует согласованность.

2.9 Требования к наблюдаемости

2.9.1 События для логирования (аудит)

– `BookLoaned` — атрибуты: loanId, readerId, copyId, loanDate, dueDate, librarianId, timestamp.

– `BookReturned` — loanId, copyId, returnDate, fineAmount, librarianId.

– `ReaderRegistered` — readerId, libraryCardNumber, timestamp.

– `FineCharged` — fineId, loanId, amount, timestamp.

– Ошибки валидации и бизнес-ошибки — тип ошибки, входные данные (обезличенные), timestamp.

2.9.2 Метрики

– Количество выдач за день / час.

– Среднее время выполнения операции выдачи.

– Количество просроченных выдач на текущую дату.

– Количество ошибок по типам.

2.9.3 Корреляционный идентификатор

Для сквозной диагностики каждый входящий запрос должен иметь уникальный `correlationId` (например, генерируется на входе или передаётся в заголовке). Этот идентификатор пробрасывается через все слои, логируется и передаётся в события. Позволяет связать логи контроллера, сервиса и БД.

ЗАКЛЮЧЕНИЕ

Разработанный комплект UML-моделей и сопутствующих артефактов полностью соответствует требованиям практических занятий №5 и №6. Модели покрывают ключевые функциональные сценарии, статическая структура согласована с прецедентами, взаимодействия детализированы до уровня сообщений и контрактов, учтены требования к архитектуре (слои, транзакции, наблюдаемость). Полученные результаты могут служить основой для проектирования API, реализации и тестирования системы автоматизации библиотеки.